Modèles Algorithmiques Fondamentaux sur Structures à Accès Direct

Thème principal : Algorithmique et Structures de Données - Patrons de Conception

Synthèse exécutive : Ce cours pose les bases fondamentales de la conception algorithmique rigoureuse en BUT 1 Informatique, indispensables pour la réussite des épreuves théoriques et des SAE (Situation d'Apprentissage et d'Évaluation). Avant toute écriture de code ou de pseudo-code, l'informaticien doit analyser le contrat d'interface et le mécanisme d'accès de sa collection de données (Direct, Séquentiel ou Dynamique). Le cours formalise trois modèles algorithmiques canoniques réutilisables sur structures à accès direct (tableaux, chaînes de caractères) : (1) le parcours complet avec traitement systématique, (2) le parcours complet avec traitement conditionné, et (3) la recherche de première occurrence. Il insiste particulièrement sur la gestion exacte des bornes d'indices (de 0 à N-1 pour éviter le dépassement de capacité 'out of bounds'), l'adoption du principe de pessimisme algorithmique, l'obligation stricte des deux conditions d'arrêt dans un algorithme de recherche, et l'analyse comparative des deux variantes d'implémentation de la recherche selon que l'objectif soit d'obtenir la position exact d'un élément ou une écriture syntaxique épurée.

1. Définition et Typologie des Mécanismes d'Accès aux Données

Avant de concevoir le moindre algorithme, le rôle de l'informaticien est d'analyser la structure de données sous-jacente. Si la collection propose un accès direct (comme un tableau de N éléments ou une chaîne de caractères), tout élément situé à l'indice i est accessible immédiatement via la syntaxe T[i]. En revanche, sur un flux séquentiel (fichier texte brut, fichier CSV lu ligne à ligne), il est matériellement impossible de lire directement la case 50 : il faut exécuter 50 fois l'opération 'suivant' depuis le début. Tenter d'écrire une instruction d'accès direct T[i] sur un fichier séquentiel est une faute conceptuelle grave qui invalide totalement l'algorithme.

- Structure de données : Collection organisée d'éléments de même type (ex: tableaux, chaînes de caractères).
- Accès Direct : Capacité d'atteindre instantanément n'importe quel élément via son indice ou sa clé (ex: T[i]) en temps constant O(1), sans visiter les éléments précédents.
- Accès Séquentiel : Contrainte imposant de démarrer au début de la collection et de progresser pas à pas (début, suivant, fin) sans saut direct possible (ex: fichier CSV brut, fichier texte, liste chaînée simple).
- Accès Dynamique : Structure hybride offrant simultanément des primitives d'accès séquentiel et d'indexation directe.
- Compatibilité structurelle : Règle stricte interdisant l'application d'un algorithme à accès direct sur un flux séquentiel.

Cas concret enseignant : *Piège classique de partiel souligné par l'enseignant : écrire une notation par indice T[i] pour traiter un fichier CSV brut ou un fichier texte ouvert. Comme le système d'exploitation gère ces données de façon séquentielle, l'algorithme est sémantiquement faux et se voit attribuer la note zéro.*

2. Premier Modèle : Parcours Complet avec Traitement Systématique

Ce modèle s'impose lorsque chaque élément d'une zone définie doit être modifié, affiché ou calculé de façon identique (ex: initialiser un tableau à zéro, afficher toutes les cases, passer chaque lettre d'une chaîne en majuscule). Le schéma canonique s'écrit : Pour i allant de 0 à N - 1 Faire Traiter(Collection[i]) Fin Pour Le nombre d'itérations étant déterminé par la taille de la zone à parcourir, l'utilisation de la boucle bornée 'Pour' est obligatoire.

- Objectif : Appliquer un traitement uniforme et inconditionnel à 100% des éléments situés entre une borne initiale et une borne finale.
- Structure itérative : Boucle bornée 'Pour' (car le nombre d'itérations est connu avant le démarrage : Fin - Début + 1).
- Gestion stricte des bornes : Pour un tableau de N éléments indicé à partir de 0, la variable de boucle i varie strictement de 0 à N - 1.
- Erreur critique 'Out of Bounds' : Faire varier l'indice jusqu'à N provoque un débordement mémoire fatal.

Cas concret enseignant : *Exemple concret donné par le professeur en amphi : 'Afficher l'intégralité d'un tableau de N cases'. L'erreur la plus fréquente des étudiants débutants est d'écrire 'Pour i allant de 0 à N', ce qui provoque un plantage immédiat (crash out of bounds) lors de la tentative d'accès à la case T[N] qui n'existe pas.*

3. Second Modèle : Parcours Complet avec Traitement Conditionné

Ce modèle conserve l'exhaustivité de l'exploration du modèle 1, mais encapsule l'action de traitement dans une condition logique. Il répond aux problèmes de comptage sélectif, de filtrage d'affichage ou d'accumulation conditionnelle. Le schéma canonique s'écrit : Pour i allant de 0 à N - 1 Faire Si Propriété(Collection[i]) Alors Traiter(Collection[i]) Fin Si Fin Pour Si la condition compare un élément à son voisin précédent (T[i-1]), les bornes doivent être ajustées (ex: démarrer i à 1 au lieu de 0) pour empêcher toute tentative de déréférencement hors limites.

- Objectif : Inspecter l'ensemble des éléments de la borne de début à la borne de fin, mais ne déclencher le traitement que si l'élément vérifie une propriété ou un filtre spécifique.
- Structure algorithmique : Imbrication d'une structure alternative 'Si ... Alors' à l'intérieur d'une boucle bornée 'Pour'.
- Complexité des conditions : Le test peut porter sur une valeur simple (ex: T[i] > 0), être composite (ex: T[i] > 0 ET T[i] mod 2 = 0) ou comparer l'élément courant à ses voisins (ex: T[i] > T[i-1]).

Cas concret enseignant : *L'analogie de l'usine de contrôle qualité : Sur une chaîne de montage, chaque pièce défile devant l'inspecteur (parcours complet systématique), mais l'inspecteur n'appose le coup de tampon de rebut (traitement conditionné) que si la pièce présente un défaut précis.*

4. Troisième Modèle : Recherche de Première Occurrence et Ses Variantes

Contrairement aux parcours complets, un algorithme de recherche doit être conçu de façon 'pessimiste' : on part du principe que l'élément n'existe pas dans la structure, et on adapte notre certitude au fil du parcours. Rédiger un algorithme de recherche avec une seule condition d'arrêt est une erreur théorique grave. Structure générale des deux variantes : - Variante 1 : S'interrompt directement dès le succès. Très performante lorsqu'il faut retourner l'indice exact de la première occurrence. - Variante 2 : Utilise une condition de boucle unique du type `Tant que (i < N et NON trouve)`. L'indice i subissant une ultime incrémentation avant la réévaluation de la boucle, il dépasse de 1 la position exacte de l'élément trouvé.

- Objectif : Localiser le tout premier élément d'une collection vérifiant une propriété donnée et interrompre l'exploration immédiatement dès qu'il est trouvé.
- Pessimisme algorithmique : Initialisation obligatoire d'un indicateur booléen 'trouve' à Faux avant d'entamer la recherche.
- Itération non bornée : Utilisation de la boucle 'Tant que' car le nombre d'itérations est imprévisible.
- Double condition d'arrêt obligatoire : (1) Succès (trouve = Vrai), (2) Échec par épuisement de la collection (Fin de structure atteinte).
- Variante 1 (Rupture) : Interrompt immédiatement la boucle dès la découverte ; conserve l'indice i exactement positionné sur l'élément.
- Variante 2 (Tant que global) : Regroupe la double condition en tête ('Tant que i < N et NON trouve') ; plus élégante syntaxiquement mais nécessite un réajustement d'indice (i - 1) si l'on veut récupérer la position.

Cas concret enseignant : *Devise de l'enseignant : 'En informatique, soyez pessimistes ! Partez du principe que ce que vous cherchez n'est pas là, et ne faites basculer votre variable qu'au moment exact où vous le touchez.' Il rappelle qu'un algorithme de recherche présenté avec une seule condition d'arrêt en examen est sanctionné par zéro à la question.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Est-ce qu'on peut appliquer le parcours complet avec traitement systématique sur une structure à accès dynamique ?
Réponse du professeur : Oui, absolument. Une structure à accès dynamique propose à la fois les mécanismes d'accès séquentiel et d'accès direct. On peut donc parfaitement appliquer un modèle conçu pour l'accès direct en exploitant la primitive d'indexation fournie par la structure.

Question étudiant : Pourquoi s'embêter à apprendre deux variantes différentes pour la recherche de première occurrence ?
Réponse du professeur : Chaque variante répond à un besoin spécifique : la Variante 1 est idéale si le problème exige de conserver la position (l'indice) exacte de l'élément trouvé sans calcul complémentaire. La Variante 2 offre une structure syntaxique plus standard et épurée (un 'Tant que' global en tête), parfaite si l'on recherche uniquement une confirmation booléenne d'existence sans soucis de position.

Question étudiant : Comment fait-on si la condition dans un parcours conditionnel dépend de l'élément précédent ou suivant ?
Réponse du professeur : On conserve le même modèle général, mais il faut impérativement réajuster les bornes de la boucle 'Pour'. Par exemple, si l'on évalue `T[i] > T[i-1]`, la boucle doit démarrer à `i = 1` jusqu'à `N - 1` pour éviter de déréférencer `T[-1]` qui provoquerait une erreur hors limites.

Question étudiant : Si l'on souhaite récupérer TOUS les indices correspondant à une valeur (par exemple trouver toutes les occurrences du nombre 5 dans un tableau), quel modèle doit-on utiliser ?
Réponse du professeur : Attention au piège ! Dès lors qu'on cherche TOUTES les occurrences et non la première, on ne fait plus une 'Recherche de première occurrence', mais un 'Parcours complet avec traitement conditionné' où le traitement consiste à stocker ou afficher l'indice i chaque fois que la valeur correspond !

■ Exercice & Démonstration d'Application

Énoncé : Écrire en pseudo-code algorithmique rigoureux l'algorithme permettant d'afficher tous les nombres strictement positifs d'un tableau d'entiers 'T' contenant 'N' éléments, en explicitant le modèle retenu et la gestion des bornes.

Résolution pas à pas :
Étape 1 : Analyse du problème et choix du modèle. On doit inspecter l'intégralité des cases d'un tableau (structure à accès direct) de la première à la dernière. L'action d'affichage ne doit s'exécuter que si la valeur respecte le critère 'strictement supérieur à 0'. Le modèle canonique est donc le 'Parcours complet avec traitement conditionné'. Étape 2 : Détermination des bornes. En indexation standard (commençant à 0), pour N éléments, les indices s'étendent de 0 à N - 1. Borne initiale = 0, Borne finale = N - 1. Étape 3 : Structure de contrôle itérative. Nombre d'itérations connu -> Boucle bornée 'Pour i allant de 0 à N - 1'. Étape 4 : Filtre logique et action. À chaque tour, évaluer 'Si T[i] > 0 Alors' et exécuter 'Afficher(T[i])'.

Résultat : Algorithme AffichagePositifs Données T : Tableau d'entiers N : Entier Variables i : Entier Début Pour i allant de 0 à N - 1 Faire Si T[i] > 0 Alors Afficher(T[i]) Fin Si Fin Pour Fin Commentaire : Cet algorithme garantit le parcours complet exhaustif sans risque de débordement d'indice (arrêt strictement à N-1) et filtre sélectivement les valeurs strictement positives.

■ Auto-Évaluation Flash d'Examen

Q : Quel modèle algorithmique doit-on utiliser pour vérifier si un tableau d'entiers contient au moins une fois la valeur 10 ?

R : La Recherche de Première Occurrence, car dès que la valeur 10 est rencontrée la première fois, la recherche s'interrompt immédiatement sans parcourir le reste du tableau.

Q : Pourquoi l'indice de fin d'un parcours complet sur un tableau de N éléments est-il égal à N - 1 et non à N ?

R : Parce qu'en informatique les indices démarrent à 0. Les N éléments occupent donc les positions 0, 1, 2, ..., N-1. Tenter d'accéder à l'indice N provoque une erreur fatale de dépassement de capacité (out of bounds).

Q : Combien de conditions d'arrêt minimales doit comporter la structure itérative d'un algorithme de recherche de première occurrence ?

R : Deux conditions d'arrêt minimales et obligatoires : (1) Le succès de la recherche (l'élément recherché a été trouvé), (2) L'épuisement de la structure (tous les éléments ont été parcourus sans succès).